Implementation of a CPU-Based ASCII 3D Rasterization Pipeline

Sahil Paliwal

March 2026

Abstract

This paper describes the design and implementation of a real-time 3D ASCII renderer built in Python. The renderer displays a rotating cube in a terminal-style window by implementing a minimal software graphics pipeline including Euler rotation, perspective projection, back-face culling, surface parameterization, and depth buffering. All geometric computations are performed in pure Python without the aid of a 3D graphics API. The visual output is produced by filling a character grid with ASCII symbols, with each face of the cube assigned a distinct character for surface differentiation. The paper documents the mathematical foundations underlying each pipeline stage and explains every implementation decision in exhaustive detail to serve as a complete reconstruction reference.

Contents

Glossary **3**

1 Introduction **5**

2 Rendering Pipeline Overview **6**

3 Mathematical Foundations **7**
3.1 Rotation Matrices . . . 7
3.2 Back-Face Culling . . . 8
3.3 Perspective Projection . . . 8
3.4 Depth Ordering . . . 8
3.5 Surface Parameterization: Formal Statement . . . 8

4 Implementation **9**
4.1 Display Architecture . . . 9
4.2 Scene Representation and Surface Parameterization . . . 10
4.2.1 What a Parameterization Is . . . 10
4.2.2 Sampling the Surface . . . 11
4.3 Coordinate Systems and the Transformation Pipeline . . . 11
4.4 Rotation . . . 12
4.5 Back-Face Culling . . . 12
4.6 Perspective Projection . . . 13
4.7 The Z-Buffer and Depth Resolution . . . 13
4.8 Rendering to the Screen . . . 14
4.9 The Main Loop . . . 15
4.10 Debug Overlay . . . 15
4.11 Interactive Controls . . . 15

5 Results **16**

6 Limitations **18**

7 Future Work **18**

8 Conclusion **19**

Glossary

ASCII American Standard Code for Information Interchange. A character encoding that assigns a number to 128 printable and control characters. In this renderer, symbols such as `@`, `|`, `~` are used as visual pixels.

Back-face culling The process of skipping any face whose outward normal points away from the camera. Since the inside of a solid object is invisible, these faces need not be drawn.

Character buffer A 2D array of characters, one entry per cell in the character grid, that accumulates the output of the rendering pipeline before being drawn to the screen.

Character grid The logical subdivision of the window into fixed-size cells, each holding one ASCII character. This is the rendering resolution of the program, and is coarser than the pixel resolution of the window.

Dot product An operation on two vectors returning a single scalar. For unit vectors it equals the cosine of the angle between them. Used here to test whether a face normal is aligned with the view direction.

Euler angles A representation of 3D rotation as three sequential rotations about the x, y, and z axes. Used in this renderer to accumulate the cube's orientation across frames.

Face normal A vector perpendicular to a surface, pointing outward away from the object. Used in back-face culling to determine face orientation relative to the camera.

Framebuffer A region of memory holding the colour of every pixel on the screen. The graphics system reads this memory to produce the final image. In this renderer the character buffer plays the equivalent role.

Object space The coordinate system local to an object, centred at the object's own origin. Object-space coordinates do not change as the object rotates — rotation transforms them into world space.

Parameterization A function $P(u, v)$ mapping 2D coordinates to a 3D point on a surface. Used here to generate sample points across each cube face.

Perspective projection A transformation mapping a 3D point to a 2D screen position by dividing x and y by the depth z. Produces foreshortening: distant objects appear smaller.

Rasterization The conversion of continuous 3D geometry into discrete grid cells (pixels or character cells).

Rotation matrix A 3×3 matrix that rotates a vector by a given angle about a given axis without changing the vector's length.

Screen space The 2D coordinate system of the character grid. After perspective projection, every 3D point has been mapped to a (col, row) address in screen space.

World / Camera space The coordinate system after object rotations have been applied. In this renderer the camera does not rotate, so world space and camera space coincide. The camera position is subtracted from world-space coordinates to produce camera-space coordinates.

Z-buffer A 2D array parallel to the character buffer storing one depth value per cell. Determines which of several overlapping geometry samples is closest to the camera and therefore visible.

Z-fighting A visual flickering artifact caused when two surfaces at nearly identical depths alternate in the depth test across frames. Avoided here by using face-centre depth rather than per-point depth.

1 Introduction

Modern graphics hardware executes a well-defined pipeline to transform 3D geometry into screen images: vertices are transformed by matrices, invisible faces are culled, geometry is projected, and pixels are shaded and depth-tested. These stages operate at enormous speed on dedicated GPU hardware, making the underlying mathematics invisible to most users.

This project reconstructs that pipeline from scratch in software, operating on a grid of ASCII characters. The goal is to produce a renderer simple enough for every component to be understood and documented individually while still implementing the same logical stages as a real graphics engine.

The result is an interactive program in which a cube rotates in real time, each face filled by a distinct ASCII character. The user can adjust the rotation speed per axis, translate the camera, and change the zoom at runtime. Despite its simplicity, the program embodies genuine graphics concepts such as coordinate transformations, perspective division, surface normal computation, dot-product visibility testing, and z-buffer depth resolution that mirrors large-scale 3D rendering pipeline.

This paper is organised as follows — Section 2 provides a high-level overview of the rendering pipeline and data flow; Section 3 derives the mathematical foundations of each stage; Section 4 documents the full implementation; Section 5 presents results; Sections 6 and 7 discuss limitations and future directions; Section 8 concludes.

2 Rendering Pipeline Overview

The renderer processes geometry through a fixed sequence of stages, each taking the output of the previous as its input. Figure 1 shows the complete data flow.

Figure 1: *Complete rendering pipeline. Data flows top to bottom; each stage operates on the output of the previous.*

Stage 1 — Face Definition. The cube's six faces are defined as a list. Each entry specifies the face's outward normal, display character, and parameterization function. No explicit vertex or edge data is stored.

Stage 2 — Back-Face Culling. Before any per-point work begins, the renderer checks whether the entire face is visible. The normal is rotated into world space and compared against the view direction via the dot product. Faces pointing away from the camera are skipped, saving the cost of sampling their 1600 surface points.

Stage 3 — Surface Sampling. Two loop variables iterate through [−size, size] and are passed through parameterization to produce 3D points in object space.

Stage 4 — Rotation. Each point is rotated by the current Euler angles (A, B, C) through three sequential rotation operations, transforming it into world space.

Stage 5 — Camera Transform. The camera position is subtracted from the rotated point. Points behind the camera ($z \leq 0$) are discarded.

Stage 6 — Perspective Projection. The 3D point is mapped to a 2D grid cell by dividing x and y by z. A zoom factor scales the result.

Stage 7 — Depth Test. The projected cell is looked up in the z-buffer. If no geometry has written there yet, or if the current face is closer, both buffers are updated.

Stage 8 — Frame Output. The character buffer is walked; every non-space character is blitted to the corresponding pixel position.

3 Mathematical Foundations

This section derives the mathematics underlying each pipeline stage.

3.1 Rotation Matrices

A 3D rotation is a linear transformation that changes a vector's direction without changing its length. Rotation about each coordinate axis is represented by a 3×3 matrix.

Rotation about the x-axis by angle A:

$$R_x(A) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos A & -\sin A \\ 0 & \sin A & \cos A \end{pmatrix}$$

Rotation about the y-axis by angle B:

$$R_y(B) = \begin{pmatrix} \cos B & 0 & \sin B \\ 0 & 1 & 0 \\ -\sin B & 0 & \cos B \end{pmatrix}$$

Rotation about the z-axis by angle C:

$$R_z(C) = \begin{pmatrix} \cos C & -\sin C & 0 \\ \sin C & \cos C & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Note: The formal geometric derivations of these standard transformation matrices are omitted for brevity, as this paper focuses strictly on their applied implementation.

The three rotations are applied sequentially — first x, then y, then z — which is equivalent to the composed matrix:

$$R = R_z(C)\ R_y(B)\ R_x(A).$$

Rotation matrices are orthogonal ($R^\top R = I$), so they preserve vector length. A unit normal therefore remains unit-length after rotation, though floating-point accumulation over many frames may introduce slight drift — this is why `normalize()` is called before each culling test.

3.2 Back-Face Culling

A face is visible only if its outward normal has a component pointing toward the camera. With the camera looking along $\mathbf{v} = (0, 0, -1)^\top$, the visibility condition for rotated unit normal $\hat{n}$ is:

$$\hat{n} \cdot \mathbf{v} > 0.$$

Since $\mathbf{v} = (0, 0, -1)^\top$ the dot product reduces to $-n_z$, so the face is visible when $n_z < 0$. The implementation uses ≤ 0 to also discard edge-on faces. For a convex object exactly three of the six faces satisfy this condition at any orientation, halving the rasterization workload.

3.3 Perspective Projection

The pinhole camera model maps a point (x, y, z) in camera space to a 2D screen position by similar triangles:

$$x_{\text{screen}} = f \cdot \frac{x}{z}, \qquad y_{\text{screen}} = f \cdot \frac{y}{z},$$

where f is focal length (approximated by `zoom`). A point at larger z maps closer to the screen centre — the mathematical origin of foreshortening. In the implementation the screen origin is centred and the y-axis flipped (screen rows increase downward):

$$x_p = \frac{\text{cols}}{2} + \text{zoom} \cdot \frac{x}{z}, \qquad y_p = \frac{\text{rows}}{2} - \text{zoom} \cdot \frac{y}{z}.$$

3.4 Depth Ordering

Let $d(f)$ denote the depth of face f (the z-coordinate of its centre in camera space). For faces $\{f_1, \ldots, f_n\}$ all projecting to the same cell, the visible face is:

$$f^* = \arg\min_i \; d(f_i).$$

This is maintained incrementally by the z-buffer: each face write checks whether its depth is smaller than the stored value and overwrites if so.

3.5 Surface Parameterization: Formal Statement

Each face is a bounded planar region described by:

$$P\colon U \subset \mathbb{R}^2 \longrightarrow \mathbb{R}^3, \qquad U = [-s,\, s] \times [-s,\, s],$$

where one coordinate is held fixed at $\pm s$ and the other two vary:

Face	Fixed coord	$P(u, v)$
Front	$z = -s$	$(u,\ v,\ -s)$
Back	$z = +s$	$(-u,\ v,\ +s)$
Right	$x = +s$	$(+s,\ v,\ u)$
Left	$x = -s$	$(-s,\ v,\ -u)$
Top	$y = +s$	$(u,\ +s,\ v)$
Bottom	$y = -s$	$(u,\ -s,\ -v)$

The sign choices on certain entries ensure traversal is consistent with the face's outward normal direction.

4 Implementation

This section documents every logical component of the renderer in enough detail that the program could be reconstructed from scratch using only this section as a reference. All geometric computation is performed in pure Python with no 3D graphics API or matrix library.

4.1 Display Architecture

A standard graphics program writes colour values into a **pixel framebuffer** — a flat array where each entry corresponds to one screen pixel. This renderer does not work that way. Instead, the screen is treated as a grid of fixed-size character cells.

The window is 1200×900 pixels. A monospaced Courier font at size 12 is used, meaning each character occupies approximately a 12×12 pixel square. Dividing the window dimensions by the font size gives:

```
cols = WIDTH  // font_size    # 1200 // 12 = 100 columns
rows = HEIGHT // font_size    #  900 // 12 =  75 rows
```

Character grid dimensions derived from window and font size

Every rendering operation targets a cell (row, col) in this grid, not an individual pixel. At the end of each frame, the grid is walked and each non-space character is drawn at pixel position $(\text{col} \times \text{font_size},\ \text{row} \times \text{font_size})$.

The effective rendering resolution is therefore 100×75, not 1200×900. The ASCII characters themselves become the visual medium — different characters on different faces create the illusion of surface differentiation.

Two parallel arrays are allocated and cleared at the start of each frame:

```
zBuffer    = [[0 for _ in range(cols)] for _ in range(rows)]
charBuffer = [[' ' for _ in range(cols)] for _ in range(rows)]
```

Z-buffer and character buffer initialisation

The **character buffer** stores which ASCII character should be drawn in each cell. The **z-buffer** stores the depth of whatever geometry last wrote to that cell. Both are described in detail in Sections 4.7 and 4.8.

4.2 Scene Representation and Surface Parameterization

The cube is not stored as a list of 8 vertices connected by 12 edges. That representation is natural for wireframe rendering, where lines are drawn between corners. This renderer is a *filled-surface renderer*: it fills each face interior with characters. A different representation is therefore used.

Each of the six cube faces is described by three components:

1. An *outward-facing surface normal* in object space — a vector perpendicular to the face, pointing away from the cube.
2. An *ASCII character* assigned to the face, giving it a distinct visual identity.
3. A *surface parameterization* — a lambda function mapping a 2D coordinate pair (u, v) to a 3D point on the face.

4.2.1 What a Parameterization Is

A flat surface in 3D can be described by two directions lying along the surface and a fixed anchor point. Any point on the surface is obtained by choosing parameter values (u, v):

$$P(u, v) = \text{anchor} + u \cdot \mathbf{d}_1 + v \cdot \mathbf{d}_2.$$

For a cube face the two directions are coordinate axes and the anchor is the fixed coordinate of that face.

Front face ($z = -\text{size}$): x and y vary freely while z is always $-\text{size}$:

```
lambda x, y: (x, y, -size)
```

The input variables map directly to the first two output slots; z is fixed.

Right face ($x = +\text{size}$): x is fixed; y and depth (z) vary freely:

```
lambda x, y: (size, y, x)
```

Here the input variable named `x` is *not* mapping to the output's x-coordinate — it maps to the output's z-coordinate. The variable is simply a number swept through a range; which slot it fills depends on the face's orientation. This is the key subtlety that makes the parameterization compact but initially unintuitive.

Left face ($x = -\text{size}$): The depth coordinate is negated to ensure the surface is traversed consistently with the outward normal:

```
lambda x, y: (-size, y, -x)
```

Table 1 gives the complete definition for all six faces.

Face	Normal	Char	Parameterization (u, v)	Fixed coord
Front	$(0,\ 0,\ -1)$	`@`	$(u,\ v,\ -s)$	$z = -s$
Right	$(1,\ 0,\ \ 0)$	`|`	$(+s,\ v,\ u)$	$x = +s$
Left	$(-1,\ 0,\ \ 0)$	`~`	$(-s,\ v,\ -u)$	$x = -s$
Back	$(0,\ 0,\ +1)$	`#`	$(-u,\ v,\ +s)$	$z = +s$
Bottom	$(0,\ -1,\ \ 0)$	`;`	$(u,\ -s,\ -v)$	$y = -s$
Top	$(0,\ +1,\ \ 0)$	`+`	$(u,\ +s,\ v)$	$y = +s$

Table 1: Cube face definitions. $s = \text{size} = 20$. The parameterization column uses (u, v) notation, corresponding to `cubeX` and `cubeY` in the source code.

4.2.2 Sampling the Surface

All points on a face are generated by a nested loop over the two free parameters:

```
size = 20
step = 1
for cubeX in range(-size, size, step):
    for cubeY in range(-size, size, step):
        i, j, k = surface(cubeX, cubeY)
```

Surface sampling loop

The variables `cubeX` and `cubeY` are the parameters (u, v). The `cube` prefix indicates that these are coordinates in the cube's local object space, as opposed to screen-space variables such as `xp` and `yp`. The name does not imply multiple cubes.

With `size=20` and `step=1`, each variable ranges from -20 to 19, producing $40 \times 40 = 1600$ sample points per face. Each resulting point (i, j, k) is a 3D location in object space, passed next to the rotation and projection stages.

4.3 Coordinate Systems and the Transformation Pipeline

Three coordinate systems are used, and it is essential to track which one each computation operates in.

Object space. Coordinates relative to the cube's own centre. A corner of a size-20 cube sits at $(20, 20, 20)$ in object space regardless of orientation. The sampling loop produces points in object space.

World space / Camera space. After rotation is applied, coordinates are in world space. The camera does not rotate, so world space and camera space coincide. The camera position $(\text{cam}_x, \text{cam}_y, \text{cam}_z)$ is subtracted from each world-space point to express it relative to the camera's eye.

Screen space. After perspective projection, a 3D point becomes a 2D address $(x_p,\ y_p)$ in the character grid.

The full transformation sequence per surface point is:

$$\underbrace{(i, j, k)}_{\text{object space}} \xrightarrow{\texttt{rotate_point}} \underbrace{(x, y, z)}_{\text{world space}} \xrightarrow{-\,\textbf{cam}} \underbrace{(x', y', z')}_{\text{camera space}} \xrightarrow{\text{perspective}} \underbrace{(x_p,\ y_p)}_{\text{screen space}}$$

4.4 Rotation

Rotation is performed by `rotate_point(i, j, k)`, which applies three sequential Euler rotations using global angle accumulators A, B, C.

Rotation about the x-axis by angle A:

$$y' = y\cos A - z\sin A, \tag{1}$$
$$z' = y\sin A + z\cos A, \tag{2}$$
$$x' = x.$$

Rotation about the y-axis by angle B:

$$x' = x\cos B + z\sin B, \tag{3}$$
$$z' = -x\sin B + z\cos B, \tag{4}$$
$$y' = y.$$

Rotation about the z-axis by angle C:

$$x'' = x'\cos C - y'\sin C, \tag{5}$$
$$y'' = x'\sin C + y'\cos C, \tag{6}$$
$$z'' = z'.$$

These are applied sequentially, each acting on the previous output — equivalent to multiplication by $R = R_z R_y R_x$.

Critically, `rotate_point` is used for *both* surface points and face normals. The culling test (Section 4.5) compares the rotated normal against the view direction; if the normal were not rotated identically to the geometry, culling would be incorrect.

The accumulators increment each frame:

```
A += rot_speed_x    # default 0.02 rad/frame
B += rot_speed_y    # default 0.02 rad/frame
C += rot_speed_z    # default 0.01 rad/frame
```

They are never reset, so the cube rotates continuously. The user can adjust each speed independently at runtime (see Section 4.11).

4.5 Back-Face Culling

Before rasterizing a face, the renderer checks whether it faces the camera. A fixed view direction $\mathbf{v} = (0, 0, -1)$ is used. After rotating the face normal and normalising to unit length $\hat{n}$:

```
nx, ny, nz = rotate_point(*normal)
n = normalize((nx, ny, nz))
if dot(n, view_dir) <= 0:
    continue
```

The `normalize()` call is essential: the dot product measures angles reliably only for unit vectors. Without it the magnitude of the rotated normal would contaminate the result, yielding incorrect culling.

Faces hidden behind closer geometry are handled by the z-buffer (Section 4.7), not by back-face culling. Back-face culling only eliminates faces that are oriented completely away from the camera.

4.6 Perspective Projection

After culling, surface points are projected onto the screen. The camera defaults to $(0, 0, -150)$. After rotating a point into world space, the camera offset is subtracted:

```
x -= cam_x    # default 0
y -= cam_y    # default 0
z -= cam_z    # default -150, so z effectively becomes z + 150
```

The default `cam_z = -150` places the camera 150 units behind the origin. Subtracting this negative value shifts all scene points forward into the camera's view.

Points at or behind the camera eye ($z \leq 0$) would cause division by zero or an inverted projection and are discarded:

```
if z <= 0:
    continue
```

Screen position is computed by perspective division:

```
xp = int(cols/2 + zoom * x / z)
yp = int(rows/2 - zoom * y / z)
```

corresponding to:

$$x_p = \frac{\text{cols}}{2} + \text{zoom} \cdot \frac{x}{z}, \qquad y_p = \frac{\text{rows}}{2} - \text{zoom} \cdot \frac{y}{z}.$$

The y-axis is negated because screen rows increase downward while the geometric y-axis points upward. Without the negation the cube appears upside down. The `cols/2`, `rows/2` offsets place the projection origin at the screen centre.

After projection, out-of-bounds points are discarded:

```
if 0 <= xp < cols and 0 <= yp < rows:
    # write to buffers
```

4.7 The Z-Buffer and Depth Resolution

After projection a point maps to cell (y_p, x_p). Multiple faces may project to the same cell — for example, the front and top faces both contribute points near the top edge of the visible cube. The z-buffer resolves which is displayed.

```
if face_depth < zBuffer[yp][xp] or zBuffer[yp][xp] == 0:
    zBuffer[yp][xp] = face_depth
    charBuffer[yp][xp] = char
```

Depth test and buffer update

A smaller `face_depth` means the face is closer to the camera. The `or zBuffer[yp][xp] == 0` clause handles the initial frame state: the z-buffer is filled with zeros at frame start. Zero is the *empty* sentinel — no real geometry has zero depth. Without this clause, the first write to any cell would always fail the `<` comparison and never succeed.

Face-centre depth. Rather than using the per-point z-value, the renderer computes one depth for the entire face from its centre point in object space:

```
cx_obj = normal[0] * size    # face centre in object space
cy_obj = normal[1] * size
cz_obj = normal[2] * size

cx, cy, cz = rotate_point(cx_obj, cy_obj, cz_obj)
cz -= cam_z                  # camera transform

face_depth = cz              # reused for all 1600 sample points
```

Face-centre depth computation

All 1600 sample points on a face share this one value. This eliminates **z-fighting** — the flickering artifact that occurs when adjacent samples on the same face alternate between winning and losing the depth test on successive frames. The trade-off is coarser depth resolution between faces, which is acceptable for a convex object where no face intersects another.

4.8 Rendering to the Screen

After `drawCube` returns, both buffers are populated. The character buffer is walked in row-major order:

```
for y in range(rows):
    for x in range(cols):
        ch = charBuffer[y][x]
        if ch != ' ':
            text = font.render(ch, True, (0, 180, 255))
            screen.blit(text, (x * font_size, y * font_size))
```

Frame output loop

Space characters are skipped — they represent empty cells where no geometry projected. Non-space characters are rendered in cyan-blue (RGB = (0, 180, 255)). The pixel position (col × font_size, row × font_size) converts grid coordinates back to pixel coordinates.

The background is cleared to solid black at the start of each frame, so the entire scene is redrawn from scratch every frame.

4.9 The Main Loop

The main loop runs at a target of 60 fps governed by `clock.tick(60)`. Each iteration:

1. **Poll keyboard state.** `pygame.key.get_pressed()` returns the state of all keys. Held keys continuously adjust rotation speeds, camera position, or zoom each frame.
2. **Clear buffers.** Z-buffer reset to zero; character buffer reset to spaces.
3. **Draw cube.** `drawCube(20, zBuffer, charBuffer)` runs the full pipeline for a cube of size 20.
4. **Blit characters.** Walk `charBuffer` and render non-space characters.
5. **Advance rotation.** Increment A, B, C by their respective speeds, unless paused.
6. **Handle events.** Process the Pygame event queue for quit, spacebar (pause toggle), and Tab (debug overlay toggle).
7. **Debug overlay.** If enabled, render current state values over the scene.
8. **Flip display.** `pygame.display.flip()` pushes the completed frame to the screen.

4.10 Debug Overlay

The debug overlay is toggled with Tab and renders the following information in green at 16 pt font in the top-left corner of the window:

- Current rotation angles A, B, C (radians)
- Per-axis rotation speeds `rot_speed_x/y/z`
- Camera position `cam_x`, `cam_y`, `cam_z`
- Current `zoom` value
- Live frames-per-second (FPS)

Because all state is accumulated and never hidden from the user, the overlay makes it possible to understand exactly what the renderer is doing at any moment — a useful property when learning how the parameters interact.

4.11 Interactive Controls

Key	Effect
`Q` / `A`	Increase / decrease x-axis rotation speed
`W` / `S`	Increase / decrease y-axis rotation speed
`E` / `D`	Increase / decrease z-axis rotation speed
Arrow keys	Translate camera left / right / up / down
Page Up / Down	Translate camera toward / away from cube
`Z` / `X`	Increase / decrease zoom (minimum: 10)
Space	Pause / resume rotation
Tab	Toggle debug overlay

Table 2: Interactive keyboard controls.

5 Results

The renderer produces a real-time display of a rotating cube at 60 fps on a standard laptop. At default settings the cube occupies approximately the central third of the 100×75 character grid.

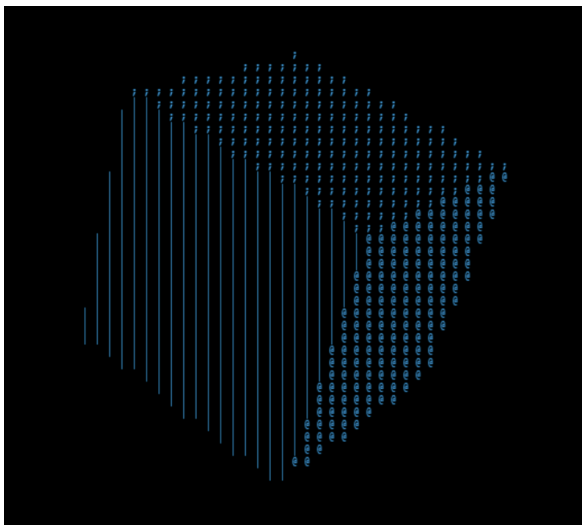
Each of the six faces is visually distinct through its assigned character:

Face	Character	Visual quality
Front	`@`	Dense, high-contrast
Right	`|`	Vertical stripe texture
Left	`~`	Light, wavy appearance
Back	`#`	Grid-like, heavy
Bottom	`;`	Light, sparse
Top	`+`	Cross-hatched

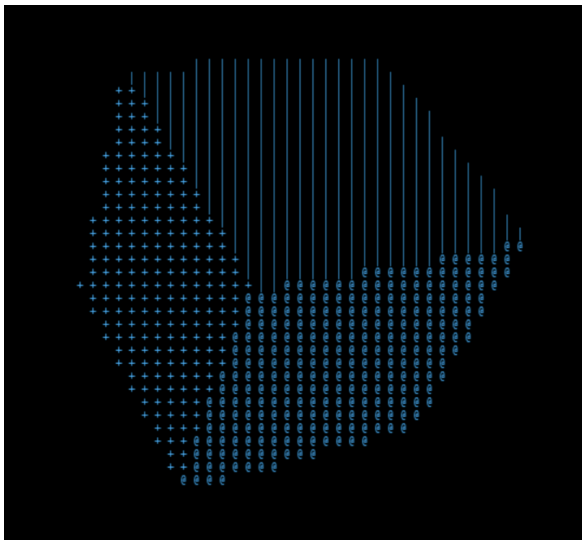
Table 3: ASCII character assignments and their visual character per face.

At default rotation speeds (`rot_speed_x` = 0.02, `rot_speed_y` = 0.02, `rot_speed_z` = 0.01 rad/frame), all three axes rotate simultaneously, producing a continuous tumbling motion that exposes new face combinations.

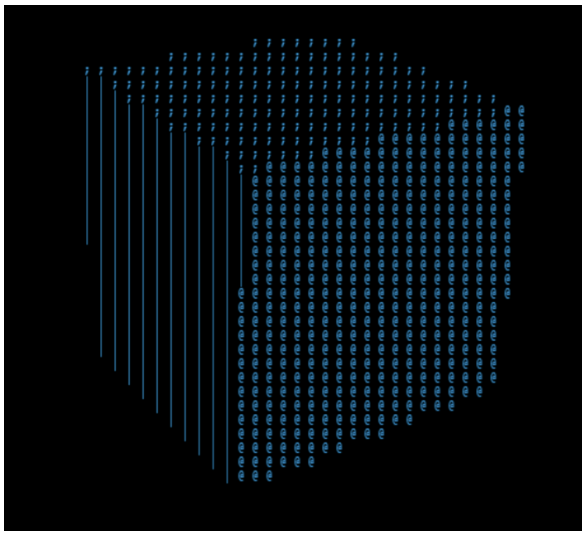
Back-face culling correctly hides the three rear faces at all orientations. The z-buffer correctly resolves depth ordering at all face boundaries with no flickering. Camera translation and zoom behave predictably; perspective foreshortening is clearly visible as the cube is brought closer via Page Down. The debug overlay confirms a stable 60 fps target.



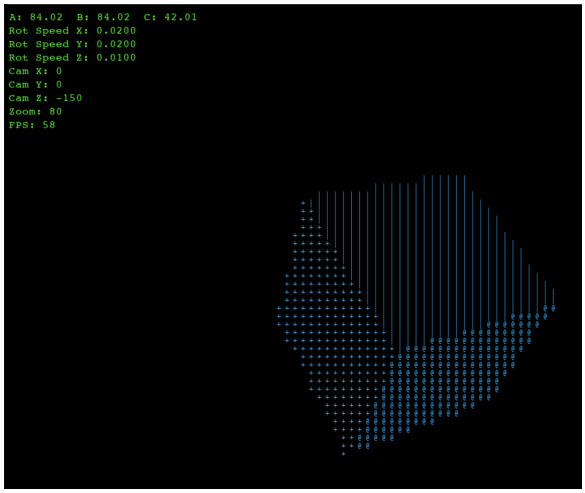
(a) Isometric view proving distinct face characters.



(b) Mid-transition proving back-face culling.



(c) Perspective foreshortening at depth.



(d) Debug overlay confirming 60 FPS target.

Figure 2: Rendering results demonstrating distinct face parameterization (top left), back-face culling during rotation (top right), and real-time performance via the debug overlay (bottom right).

6 Limitations

Character grid resolution. The rendering resolution is 100×75 cells. Diagonal edges appear stepped at this coarse resolution. Reducing the font size would increase resolution at the cost of character legibility.

Euler angle gimbal lock. The Euler representation can suffer from gimbal lock — a condition where two rotation axes align and one degree of freedom is lost. This is not a practical problem for continuous low-speed rotation, but would affect precise orientation control.

No lighting model. Real shading requires computing the angle between the surface normal and a light direction and modulating brightness accordingly. Here each face has a fixed character with no brightness variation; surface differentiation relies entirely on character density.

Clip artefacts at extreme zoom. At very high zoom values, projected points near the cube edges can exceed the grid bounds. These are correctly discarded by the bounds check, but portions of the cube may disappear at extreme settings.

Face-centre depth approximation. Using face-centre depth rather than per-point depth is an approximation that would produce incorrect results for intersecting geometry or large faces at steep viewing angles. For a convex cube it is always correct.

CPU-only performance. All computation runs on a single Python thread. Up to $3 \times 1600 = 4800$ point transformations are performed per frame. Python's interpreted overhead means the renderer does not scale well to higher resolutions or more complex scenes.

7 Future Work

Per-point depth buffering. Replacing face-centre depth with the per-point z-value from the projection stage would give accurate depth resolution for non-convex or intersecting geometry.

Lambertian shading. A Lambertian illumination model computes $\max(0,\ \hat{n} \cdot \hat{L})$, where $\hat{L}$ is a unit light direction. Mapping this value to character density — substituting dense characters (`@`) with sparser ones (`.`) for darker regions — would produce a much stronger illusion of 3D form without requiring colour output.

Multiple objects. Supporting multiple objects, each with its own model transform, requires a scene graph and per-object rotation matrices. The z-buffer already supports multi-object depth resolution without modification.

Quaternion rotation. Replacing Euler angles with quaternions eliminates gimbal lock and enables smooth interpolation between arbitrary orientations via spherical linear interpolation (slerp) — the standard approach in game engines.

NumPy vectorization. Replacing the nested Python loops with NumPy array operations would apply rotation and projection to all surface points simultaneously using compiled C code, potentially yielding a 10–100× performance improvement and making higher-resolution grids feasible.

Perspective-correct interpolation. If per-point shading or texture mapping were added, naive screen-space linear interpolation would introduce distortion. Perspective-correct interpolation divides by z at each step to correct this — the standard technique used in hardware rasterizers.

8 Conclusion

This paper has presented a complete software 3D renderer implemented in Python, producing real-time ASCII output of a rotating cube. Starting from first principles, the renderer implements the core stages of a standard graphics pipeline: surface parameterization, back-face culling via the dot product, Euler rotation via sequential rotation matrices, perspective projection via division by depth, and visibility resolution via a z-buffer.

Each stage was derived mathematically in Section 3 and documented in full implementation detail in Section 4. The result is a self-contained program in which every character cell has a traceable origin in the mathematics.

The project demonstrates that the fundamental concepts of 3D graphics are accessible without GPU hardware, specialised libraries, or high-level abstractions. A rotating solid cube with correct depth ordering and back-face culling requires only trigonometry, vector arithmetic, and two 2D arrays. Implementing these stages manually reveals a depth of understanding that using a graphics API cannot — every artifact, every edge case, and every design trade-off is directly visible in the code.

References

[1] Foley, J. D., van Dam, A., Feiner, S. K., and Hughes, J. F. (1990). *Computer Graphics: Principles and Practice* (2nd ed.). Addison-Wesley.

[2] Shirley, P. and Marschner, S. (2009). *Fundamentals of Computer Graphics* (3rd ed.). A K Peters/CRC Press.

[3] Strang, G. (2006). *Linear Algebra and Its Applications* (4th ed.). Thomson Brooks/-Cole.

[4] Scratchapixel (2014). *The Perspective and Orthographic Projection Matrix*. https://www.scratchapixel.com

[5] Scratchapixel (2014). *Rasterization: A Practical Implementation*. https://www.scratchapixel.com